

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CƠ KHÍ - BỘ MÔN CƠ ĐIỆN TỬ



NHẬP MÔN THỊ GIÁC MÁY TÍNH
BÀI TẬP LỚN

LỚP TN01 - HK 251

GVHD: PGS.TS. LÊ ĐỨC HẠNH

SINH VIÊN THỰC HIỆN:

STT	Họ và tên	MSSV
1	Võ Hữu Dư	2210604
2	Đào Trọng Chân	2210350
3	Dương Quang Duy	2210497
4	Trần Quang Đạo	2210647

TP.HCM, Ngày 23 tháng 11 năm 2025

Mục lục

1	Câu 5	2
1.1	Cơ sở lý thuyết	2
1.2	Các bước thực hiện	3
2	Câu 6	11

Danh sách bảng

1	Bảng dữ liệu tọa độ các điểm dùng để calib	6
---	--	---

Danh sách hình vẽ

1	Mặt phẳng hệ tọa độ dùng để calib camera trực tiếp	3
2	Dùng thước dây đo độ cao từ camera đến mặt phẳng hệ tọa độ	4
3	Thứ tự đánh dấu các điểm dùng để calib trên giao diện GUI	6
4	Kết quả ma trận calib thu được từ phương pháp trực tiếp	8
5	Các điểm dùng để kiểm tra lại, độ cao $h = 492\text{mm}$	9
6	Kết quả tọa độ các điểm trong không gian thực	10
7	Kết quả khoảng cách giữa hai điểm P1 và P2	10

1 Tính toán chọn vùng làm việc cho camera so sánh với thực tế và tính toán

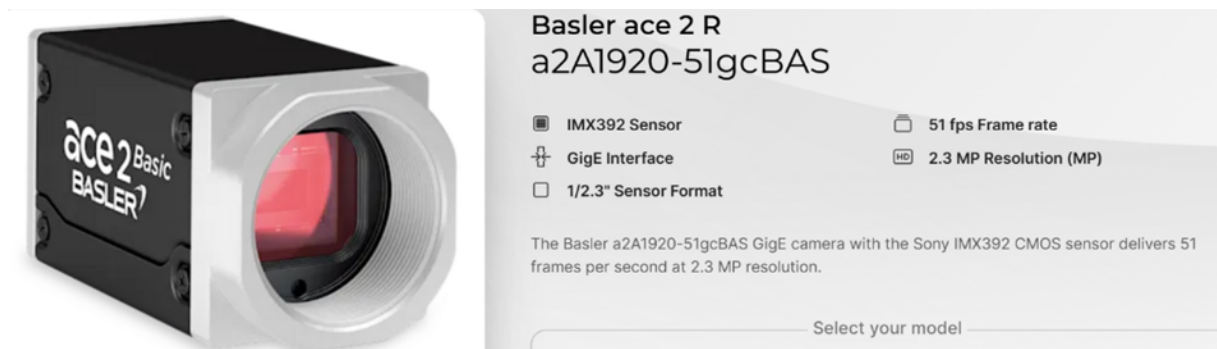
1.1 Xác định các thông số sử dụng cho tính toán

Vì camera nhìn từ trên xuống nên FOV tối thiểu thực tế = chiều dài tối thiểu x chiều rộng tối thiểu = 268mm x 164mm.

Lựa chọn camera:

Nhóm sử dụng camera Basler ace 2 R a2A1920-51gcBAS với các thông số cần sử dụng:

- Resolution: 1920x1200 pixels
- Sensor size: 6.6mm x 4.1mm
- Focal Length: 8mm

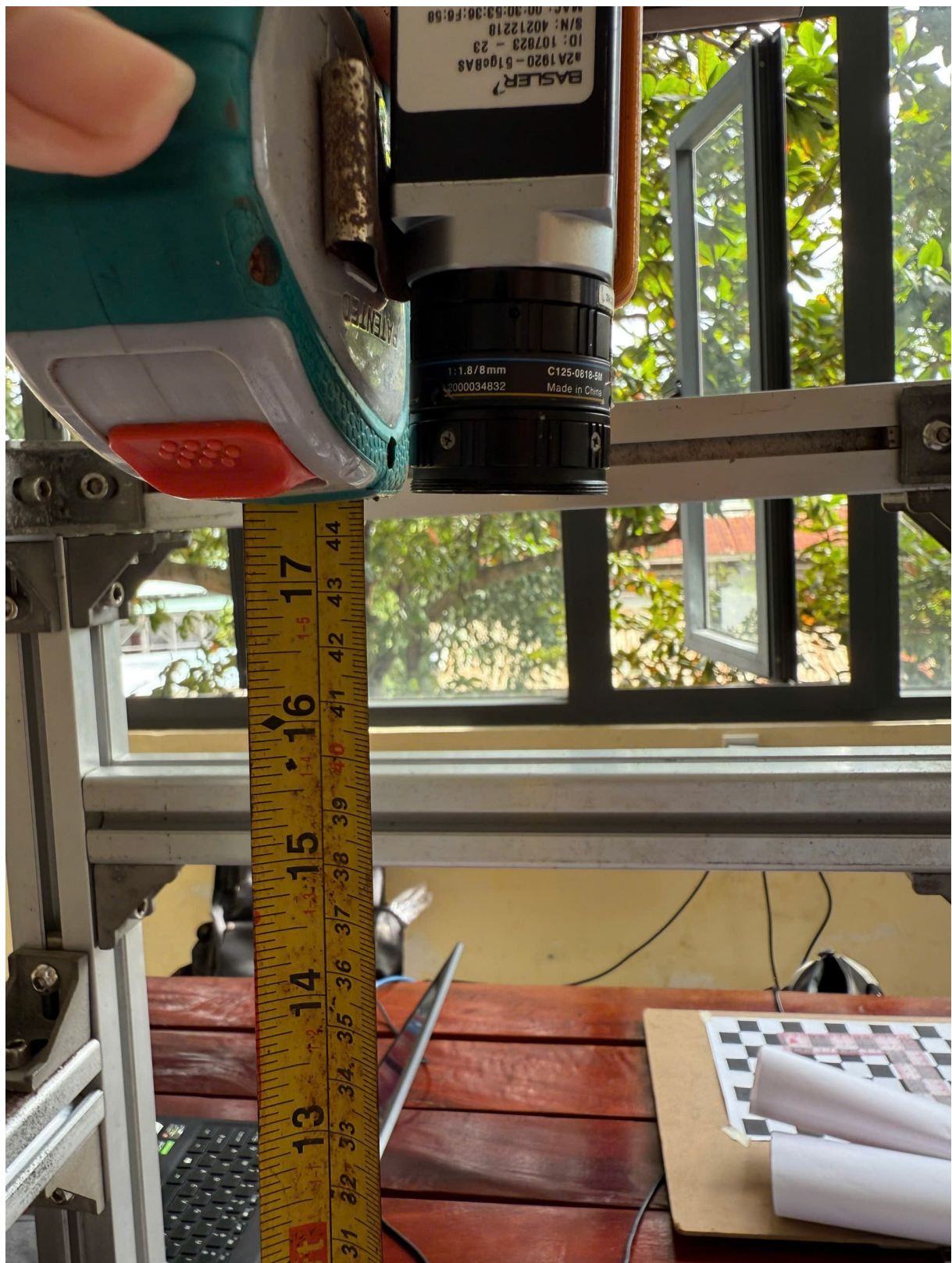


Hình 1: Thông số camera Basler a2A1920-51gcBAS



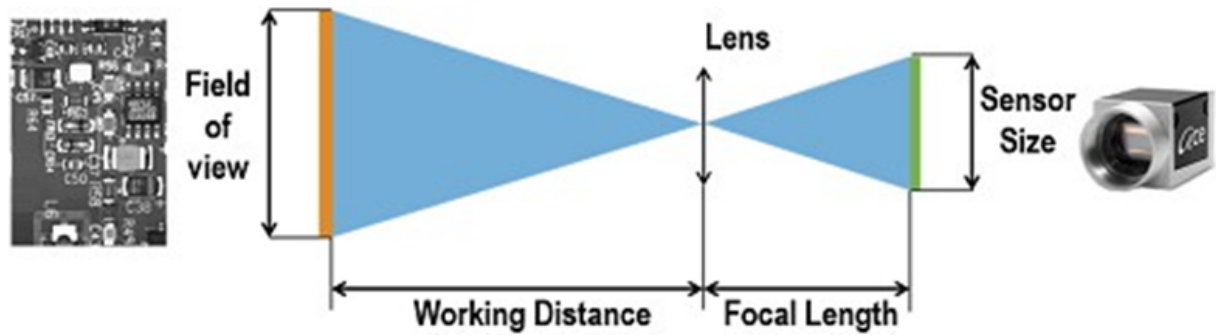
Hình 2: Hình ảnh thực tế của camera Basler a2A1920-51gcBAS

Nhóm tiến hành chọn trước Working Distance: 445 mm



Hình 3: Working Distance thực tế

Tính vùng quan sát, làm việc của camera với Working Distance đã chọn:



Hình 4: Mô tả cách thức suy ra công thức tính toán

Theo tỷ lệ đồng dạng tam giác ở hình 4 ta có công thức như sau:

$$\frac{FOV}{Working\ Distance} = \frac{Sensor\ Size}{Focal\ Length} \quad (1)$$

Suy ra:

$$FOV = \frac{Sensor\ Size \times Working\ Distance}{Focal\ Length} \quad (2)$$

1.2 Tính toán theo lý thuyết

Với Working Distance = 445mm, Focal Length = 8mm, ta tính được:
FOV theo chiều ngang:

$$FOV_h = \frac{6.6 \times 445}{8} = 367.125mm \quad (3)$$

FOV theo chiều dọc:

$$FOV_v = \frac{4.1 \times 445}{8} = 228.0625mm \quad (4)$$

1.3 So sánh với yêu cầu thực tế

- FOV tính toán được: 367.125mm x 228.0625mm
- FOV tối thiểu yêu cầu: 268mm x 164mm

Kết luận: FOV tính toán lớn hơn FOV tối thiểu yêu cầu, đảm bảo camera có thể quan sát toàn bộ vùng làm việc cần thiết.

1.4 Tính sai số của đường chéo FOV

- Đường chéo theo lý thuyết tính từ kích thước FOV ở trên:

$$\sqrt{367.125^2 + 228.0625^2} = 432.196mm \quad (5)$$

- Đường chéo FOV đo được thực tế:

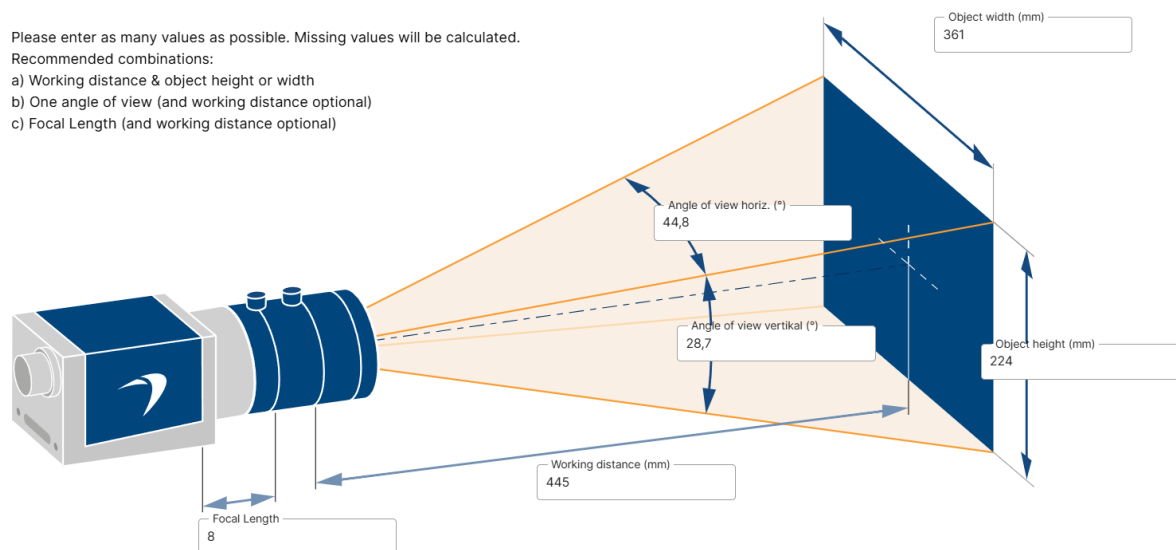
$$\sqrt{367.125^2 + 228.0625^2} = 432.196mm \quad (6)$$

- Sai số đường chéo FOV:

$$\frac{|432.196 - 430|}{432.196} \times 100\% = 0.51\% \quad (7)$$

1.5 Kiểm tra lại bằng công cụ Len Selector của nhà sản xuất

Để kiểm tra lại kết quả tính toán, nhóm sử dụng công cụ Len Selector của nhà sản xuất Basler để mô phỏng lại các thông số đã chọn. Kết quả thu được như hình bên dưới:



Hình 5: Đối chiếu các thông số bằng công cụ Len Selector của Basler

Như vậy, các kết quả tính toán tương đối khớp với kết quả từ công cụ Len Selector của nhà sản xuất Basler.



2 Câu 2



3 Câu 3

4 Câu 5

Thực hiện calib camera bằng phương pháp trực tiếp sau đó tìm lại ma trận thông số nội và ngoại, so sánh với phương pháp gián tiếp đã làm ở câu 3.

4.1 Cơ sở lý thuyết

Quá trình hiệu chỉnh camera bằng phương pháp trực tiếp là đi tìm các thông số trong ma trận chiếu (projection matrix), khi ta đã biết tọa độ các điểm $A_1, A_2, \dots, A_n$ trong hệ tọa độ vật và tọa độ tương ứng $a_1, a_2, \dots, a_n$ của chúng trên mặt phẳng ảnh.

Giả sử bỏ qua các điểm distortion, có 12 thông số cần tìm trong ma trận hình chiếu. Giả sử các điểm trong tọa độ thực là $A_i = [X_i \ Y_i \ Z_i \ 1]^T$ biết trước có ảnh $a_i = [u_i \ v_i]$ biết trước ($i = 1, 2, 3, \dots$). Ta có công thức chuyển đổi

$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \Rightarrow \begin{cases} su = m_{11}X + m_{12}Y + m_{13}Z + m_{14} \\ sv = m_{21}X + m_{22}Y + m_{23}Z + m_{24} \\ s = m_{31}X + m_{32}Y + m_{33}Z + m_{34} \end{cases} \quad (8)$$

Từ (1) rút ra biểu thức cho u và v

$$u = \frac{m_{11}X + m_{12}Y + m_{13}Z + m_{14}}{m_{31}X + m_{32}Y + m_{33}Z + m_{34}}$$

$$v = \frac{m_{21}X + m_{22}Y + m_{23}Z + m_{24}}{m_{31}X + m_{32}Y + m_{33}Z + m_{34}}$$

Khai triển và biến đổi chuyển về ta có biểu thức

$$0 = m_{11}X + m_{12}Y + m_{13}Z + m_{14} - m_{31}uX - m_{32}uY - m_{33}uZ - m_{34}u \quad (9)$$

$$0 = m_{21}X + m_{22}Y + m_{23}Z + m_{24} - m_{31}vX - m_{32}vY - m_{33}vZ - m_{34}v \quad (10)$$

Đưa về dạng ma trận $\mathbf{Ax} = \mathbf{0}$ ta có

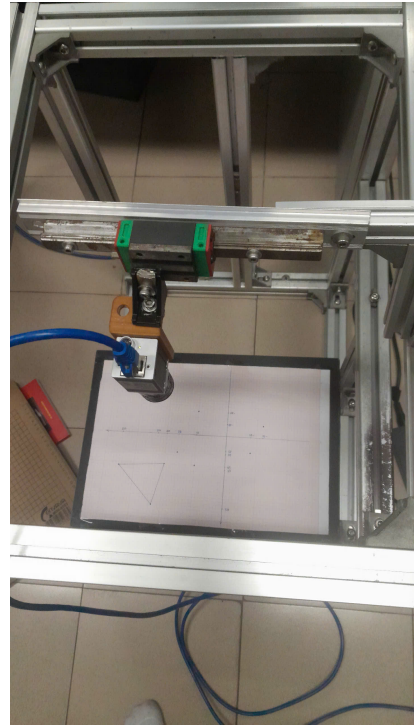
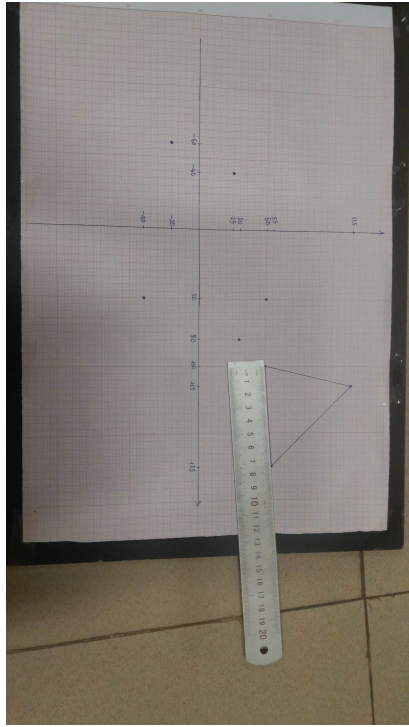
$$\begin{bmatrix} X_1 & Y_1 & Z_1 & 1 & 0 & 0 & 0 & 0 & -u_1X_1 & -u_1Y_1 & -u_1Z_1 & -u_1 \\ 0 & 0 & 0 & 0 & X_1 & Y_1 & Z_1 & 1 & -v_1X_1 & -v_1Y_1 & -v_1Z_1 & -v_1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ X_n & Y_n & Z_n & 1 & 0 & 0 & 0 & 0 & -u_nX_n & -u_nY_n & -u_nZ_n & -u_n \\ 0 & 0 & 0 & 0 & X_n & Y_n & Z_n & 1 & -v_nX_n & -v_nY_n & -v_nZ_n & -v_n \end{bmatrix} \begin{bmatrix} m_{11} \\ m_{12} \\ m_{13} \\ m_{14} \\ m_{21} \\ m_{22} \\ m_{23} \\ m_{24} \\ m_{31} \\ m_{32} \\ m_{33} \\ m_{34} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix} \quad (11)$$

Giải hệ này tìm ra các hệ số của ma trận hình chiếu sử dụng phương pháp SVD (Singular Value Decomposition). Rồi từ đó phân tách ra thành ma trận thông số nội K , ma trận xoay R và vector tịnh tiến T .

4.2 Các bước thực hiện

Quá trình thực hiện bao gồm các bước sau:

- Bước 1: Sử dụng một mặt phẳng chứa một hệ tọa độ xác định như hình, chiều dài mỗi ô trên hệ tọa độ là 5mm.



Hình 6: Mặt phẳng hệ tọa độ dùng để calib camera trực tiếp

- Bước 2: Chấm các điểm có tọa độ xác định trên mặt phẳng hệ tọa độ, ghi lại các tọa độ thực tế X_i, Y_i của các điểm này trong hệ tọa độ không gian.
- Bước 3: Dùng thước dây đo khoảng cách từ camera đến mặt phẳng hệ tọa độ để lấy giá trị Z_i cho các điểm đã chấm. Với mỗi độ cao, các điểm đã chấm trên mặt phẳng sẽ có tọa độ trong không gian là $A_i = [X_i \ Y_i \ Z_i \ 1]^T$.



Hình 7: Dùng thước dây đo độ cao từ camera đến mặt phẳng hệ tọa độ

- Bước 4: Chụp ảnh các điểm đã chấm từ camera, sử dụng code để lấy tọa độ ảnh u_i, v_i tương ứng của các điểm này trên mặt phẳng ảnh.

Vết hàm để thực hiện chấm các các điểm để lấy tọa độ pixel và xuất ra ma trận tọa độ trên không gian ảnh và không gian thực của từng điểm

Listing 1: Hàm đọc pixel từ điểm chấm

```
import cv2
import numpy as np

def create_calibration_data(image_path):
    """
    GUI creates calibration data by allowing the
    user to click on points in the image.
    Returns a list of pixel coordinates.
    """
    # Load image
    img = cv2.imread(image_path)
```

```
if img is None:
    print("Cannot load image.")
    return None

# List to store pixel coordinates
points = []

def mouse_callback(event, x, y, flags, param):
    if event == cv2.EVENT_LBUTTONDOWN:
        points.append((x, y))
        print(f"Clicked point: ({x}, {y})")
        cv2.circle(img, (x, y), 5, (0, 255, 0), -1)
        cv2.imshow('Image', img)

cv2.imshow('Image', img)
cv2.setMouseCallback('Image', mouse_callback)

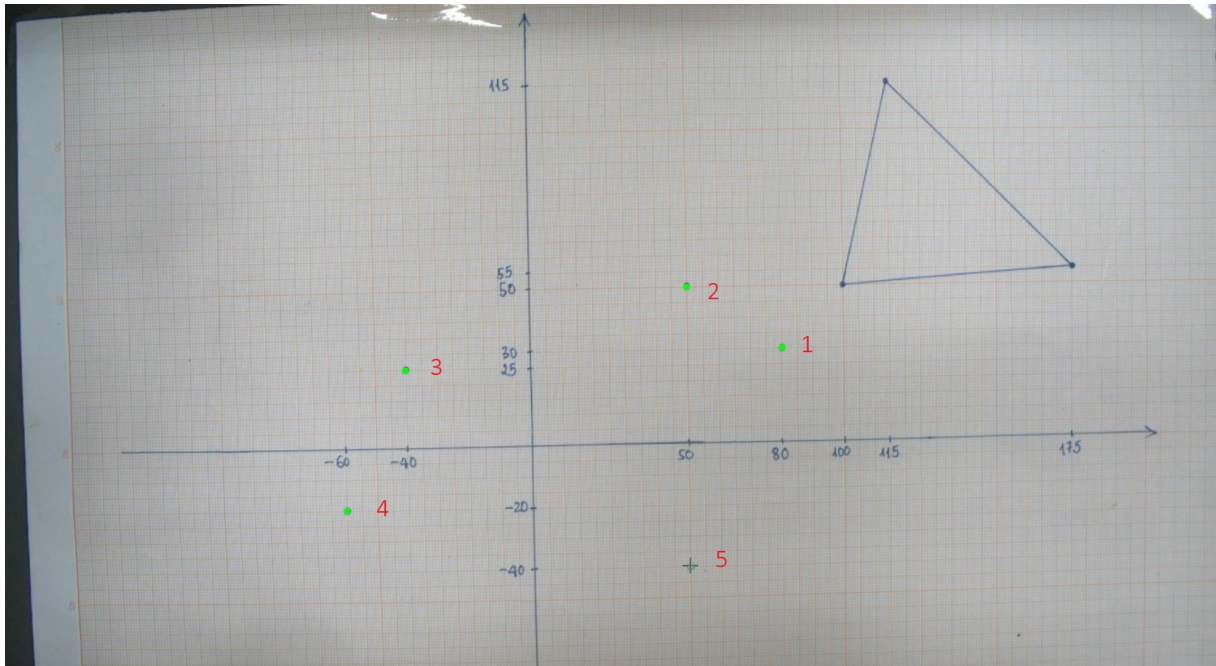
print("Click points on the image. Press 'q' to stop.")
while True:
    key = cv2.waitKey(1) & 0xFF
    if key == ord('q'):
        break

cv2.destroyAllWindows()

if not points:
    print("No points were clicked.")
    return None

return points
```

Thứ tự đánh dấu các điểm trên giao diện GUI



Hình 8: Thứ tự đánh dấu các điểm dùng để calib trên giao diện GUI

Mảng chứa dữ liệu tọa độ thực tế và tọa độ ảnh thu được sau khi chấm điểm

Bảng 1: Bảng dữ liệu tọa độ các điểm dùng để calib

u_1	v_1	x_1	y_1	z_1
1199	553	80	30	492
1057	462	50	50	492
639	587	-40	25	492
552	796	-60	-20	492
1064	880	50	-40	492
1208	552	80	30	475
1061	459	50	50	475
629	588	-40	25	475
540	804	-60	-20	475
1068	888	50	-40	475
1212	554	80	30	467
1063	458	50	50	467
624	591	-40	25	467
533	811	-60	-20	467
1070	897	50	-40	467

Sau khi có dữ liệu tọa độ thực và tọa độ ảnh, tiến hành sử dụng SVD để giải hệ phương trình và tìm ma trận chiếu. Từ ma trận chiếu ta tách ra ma trận thông số nội và ngoại.

Listing 2: Code tìm các ma trận calib

```
# Get image coordinates and world coordinates
imgCoor = data[:, :2] # columns u_1, v_1
```

```
worldCoor = data[:, 2:] # columns x_1, y_1,
                        z_1

# Build matrix P
P = np.zeros((2*len(imgCoor), 12))
j = 0

for i in range(0, 2*len(imgCoor), 2):
    P[i, 0:3] = worldCoor[j, 0:3]
    P[i+1, 4:7] = worldCoor[j, 0:3]
    P[i, 3] = 1
    P[i+1, 7] = 1
    P[i, 8:12] = -np.concatenate([worldCoor[j,
        0:3], [1]]) * imgCoor[j, 0]
    P[i+1, 8:12] = -np.concatenate([worldCoor[j,
        0:3], [1]]) * imgCoor[j, 1]
    j += 1

# SVD decomposition
U, S, V = np.linalg.svd(P)
V_last = V[-1, :]

# Create matrix M
M = np.zeros((3, 4))
M[0, 0:4] = V_last[0:4]
M[1, 0:4] = V_last[4:8]
M[2, 0:4] = V_last[8:12]

# Extract intrinsic and extrinsic matrices
# from matrix M
M = M / M[2, 3]
H = M[:, 0:3] # first 3 columns of M
T = M[:, 3]   # translation matrix

# QR decomposition to find K and Ro
Q, R = np.linalg.qr(np.linalg.inv(H))
K = np.linalg.inv(R)
Ro = Q.T

# K must have positive diagonal elements
t = np.array([[ -1, 0, 0],
               [ 0, 1, 0],
               [ 0, 0, 1]])
K = K @ t / K[2, 2]
```

Kết quả ma trận hình chiếu, ma trận thông số nội và ngoại thu được


```

Projected matrix
[[-1.66835382e-02  7.66088233e-04 -5.78996968e-03 -1.30681060e-01]
 [ 4.07786417e-04  1.69963840e-02 -3.09870761e-03 -9.91116057e-01]
 [ 1.18729419e-07  4.63444235e-07 -6.15664618e-06 -5.70505798e-04]]

Intrinsic matrix K:
[[2.71987632e+03  6.56767583e-01  8.92163113e+02]
 [0.00000000e+00  2.70743524e+03  7.08122069e+02]
 [0.00000000e+00  0.00000000e+00  1.00000000e+00]]

Extrinsics Matrix:
[[-9.99627654e-01  2.07538248e-02 -1.77153221e-02  2.29061756e+02]
 [-1.93619277e-02 -9.96963839e-01 -7.54202793e-02  1.73725852e+03]
 [-1.92267948e-02 -7.50491940e-02  9.96994458e-01  1.00000000e+00]]
    
```

Hình 9: Kết quả ma trận calib thu được từ phương pháp trực tiếp

Thực hiện kiểm tra lại kết quả ma trận calib thu được bằng cách tính toán lại tọa độ thực tế của các điểm checkpoint từ tọa độ pixel. Công thức liên hệ giữa tọa độ thực M_w và tọa độ ảnh pixel m_p là

$$M_w = R^{-1}(K^{-1}zm_p - T) \quad (12)$$

Listing 3: Kiểm tra kết quả các ma trận calib

```

# Calculate world coordinates from image
coordinates
z_known = 492 # distance from camera to
surface

image_path_test = r"D:\Fundamental\4\251\
Computer_Vision\BTL\pics\b5_new\492.jpg"

worldCoor_3D = []
# imgCoor_test = np.array([[845, 507], [1462,
100], [1750, 412]])
imgCoor_test = create_calibration_data(
    image_path_test)
imgCoor_test = np.array(imgCoor_test)
point_names = ['P1', 'P2', 'P3']

for i in range(len(imgCoor_test)):
    u = imgCoor_test[i, 0]
    v = imgCoor_test[i, 1]

# Define the system of equations
A = np.array([
    [M[0,0] - u*M[2,0], M[0,1] - u*M[2,1]],
    
```



```

[M[1,0] - v*M[2,0], M[1,1] - v*M[2,1]]
])

B = np.array([
    u*M[2,2]*z_known + u*M[2,3] - M[0,2]*
    z_known - M[0,3],
    v*M[2,2]*z_known + v*M[2,3] - M[1,2]*
    z_known - M[1,3]
])

# Solve for world coordinates (X, Y)
world_pt = np.linalg.solve(A, B)
worldCoor_3D.append([np.ceil(world_pt[0]), np.
    ceil(world_pt[1]), z_known])

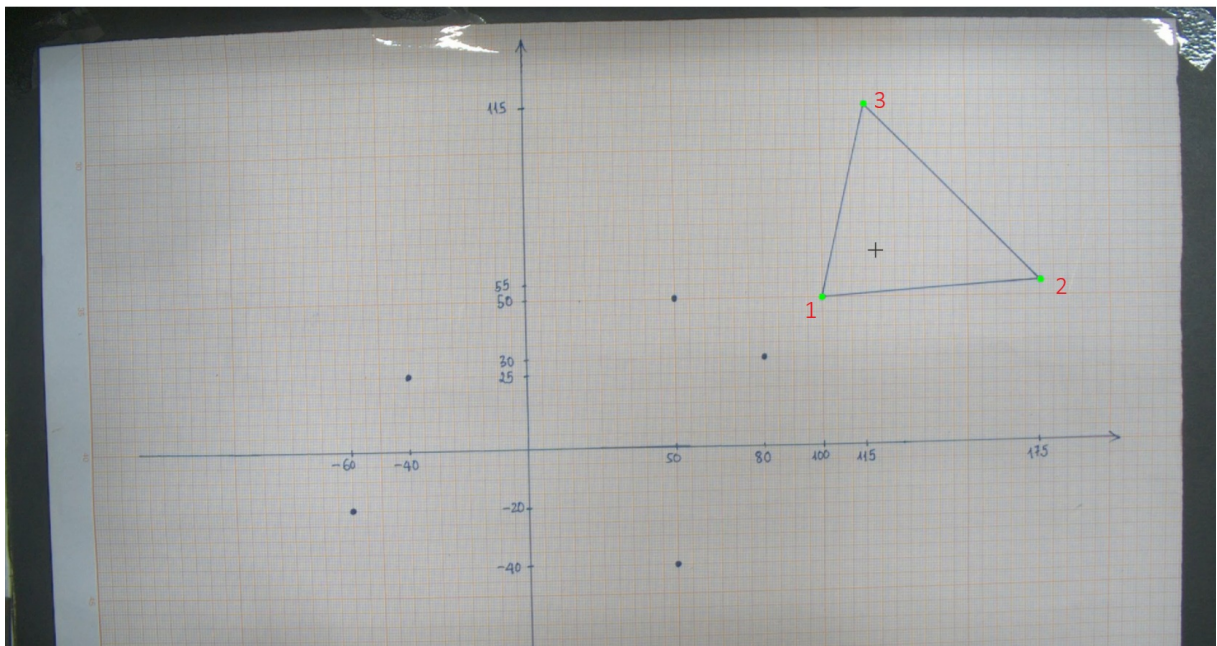
worldCoor_3D = np.array(worldCoor_3D)

# Calculate Euclidean distance between P1 and
P2
p1 = worldCoor_3D[0]
p2 = worldCoor_3D[1]

distance = np.sqrt((p2[0] - p1[0])**2 + (p2[1]
    - p1[1])**2 + (p2[2] - p1[2])**2)

```

Các điểm dùng để kiểm tra lại kết quả sau khi tính toán ma trận calib ở độ cao $h = 492mm$



Hình 10: Các điểm dùng để kiểm tra lại, độ cao $h = 492mm$

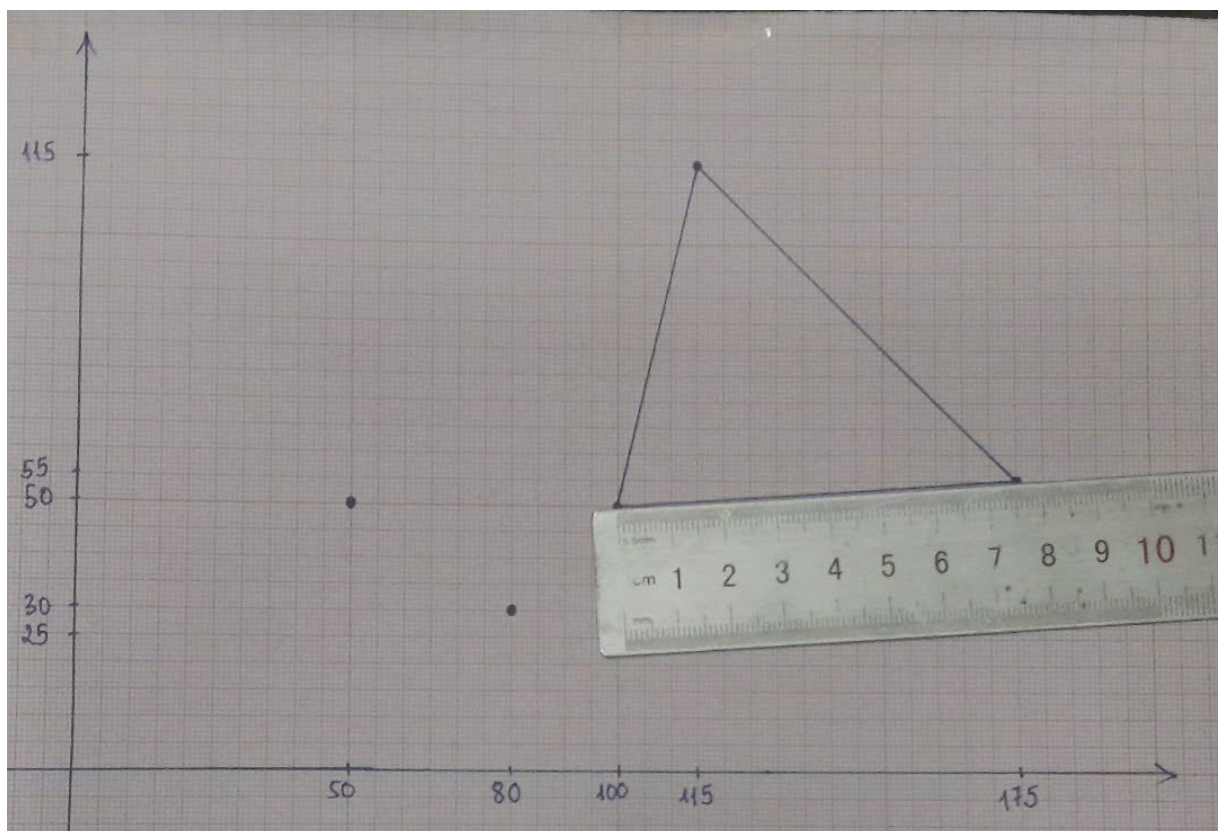
Kết quả

```
-----
Img_coordinate at P1: (1289, 459) -----> World_Coordinate: (100, 50)
Img_coordinate at P2: (1633, 431) -----> World_Coordinate: (172, 54)
Img_coordinate at P3: (1354, 155) -----> World_Coordinate: (114, 114)
-----
Distance between P1(100, 50) and P2(172, 54) with world_Coordinate: 72.111026 mm
```

Hình 11: Kết quả tọa độ các điểm trong không gian thực

So với tọa độ 3 điểm P_1, P_2, P_3 lần lượt là $(100, 50, 492)$, $(175, 55, 492)$ và $(115, 115, 492)$. Kết quả tọa độ tính toán lại từ các ma trận calib theo phương pháp trực tiếp là khá chính xác.

Khoảng cách tính toán giữa hai điểm P_1 và P_2 là 72.111026 mm, so với khoảng cách thực tế là 75 mm, kết quả thu được là khá chính xác.



Hình 12: Kết quả khoảng cách giữa hai điểm P_1 và P_2

So sánh và nhận xét: So với ma trận thông số nội K thu được từ phương pháp calib gián tiếp



5 Câu 6